

Глубинное обучение в обработке звука

Синтез речи I: акустический модели

План лекции

- Задачи синтеза речи
- Оценка качества синтеза
- Данные
- Виды синтеза речи
- Основы лингвистики
- Анализ текста
- Акустические модели

Задачи синтеза речи

Задачи синтеза речи

Синтез речи (Text – to – Speech, TTS) – это технология, которая преобразует текст в речь

Заменить человека в рутинных задачах



Упростить взаимодействие с устройствами



Помощь людям с ограниченными возможностями



Разработать новые продукты



Задачи синтеза речи

Действительно ли нужно, чтобы качество синтеза было высоким и чтобы голос звучал как человеческий?



- Люди чувствительны не только к произносимым словам, но и к тому, как они произносятся
- Большинство людей быстро находят механические голоса раздражающими

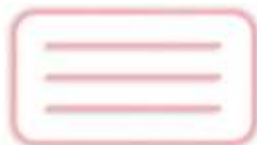
Тесты показывают: чем естественнее звучит голос, тем сильнее растёт удовлетворенность пользователей

Нужно создать систему, которая:

- Четко передает сообщение
- Делает это человеческим голосом

Задачи синтеза речи

Основными требованиями к технологии синтеза речи являются **разборчивость** и **естественность**



Разборчивость

Относительное
количество правильно
принятых слушателями
элементов речи



Естественность

Показатель близости
сгенерированной речи с
человеческой



Предпочтение

Выбор слушателями
лучшего синтеза



Понятность

Степень понимания
полученных сообщений.

Оценка качества синтеза

Оценка качества синтеза

Субъективные

Mean Opinion Score

Comparative Mean Opinion Score

Similarity Mean Opinion Score

Side By Side

Similarity Side By Side

MUSHRA

Объективные

WER/CER

Mel-cepstral Distortion

UTMOS/NISQA

Frechet Audio Distance

Speaker Similarity

Real-Time Factor/Latency

Субъективные оценки: MOS

MOS (Mean Opinion Score) рассчитывается как среднее арифметическое всех оценок слушателей:

$$MOS = \frac{1}{N} \sum_{i=1}^N r_i,$$

где N – количество оценок, r_i – оценка от i -го слушателя

Оценка r_i	Качество	Описание
1	<i>Bad</i>	Речь неразборчива, с явными дефектами
2	<i>Poor</i>	Речь разборчива, но есть заметные дефекты
3	<i>Fair</i>	Речь разборчива, но есть незначительные дефекты
4	<i>Good</i>	Хорошее качество, небольшие дефекты не мешают восприятию
5	<i>Excellent</i>	Отличное качество, без заметных дефектов

Субъективные оценки: CMOS

CMOS (*Comparative Mean Opinion Score*) используется для сравнения синтезированной речи и рассчитывается на основе сравнительных оценок слушателей:

$$CMOS = \frac{1}{N} \sum_{i=1}^N c_i,$$

где N – количество оценок, c_i – оценка от i -го слушателя, показывающая насколько одна запись лучше другой

Оценка c_i	Качество	Описание
3	Значительно лучше	Первая запись значительно лучше второй
2	Лучше	Первая запись лучше второй
1	Немного лучше	Первая запись немного лучше второй
0	Одинаково	Обе записи одинакового качества
−1	Немного хуже	Первая запись немного хуже второй
−2	Хуже	Первая запись хуже второй
−3	Значительно хуже	Первая запись значительно хуже второй

Субъективные оценки: *SMOS*

SMOS (Similarity Mean Opinion Score) используется для оценки сходства синтезированной речи с эталонной:

$$SMOS = \frac{1}{N} \sum_{i=1}^N s_i,$$

где N – количество оценок, s_i – оценка от i -го слушателя, показывающая сходство записей

Оценка s_i	Качество	Описание
1	Очень непохоже	Запись значительно отличается от оригинала
2	Непохоже	Запись имеет заметные отличия от оригинала
3	В меру похоже	Запись частично похожа на оригинал
4	Похоже	Запись с небольшими отличиями соответствует оригиналу
5	Очень похоже	Запись практически неотличима от оригинала

Субъективные оценки: SBS

SBS (Side By Side) используется для прямого сравнения двух синтезированных аудио

Пусть у нас есть два варианта синтезированной речи: A и B

Тогда SBS рассчитывается следующим образом:

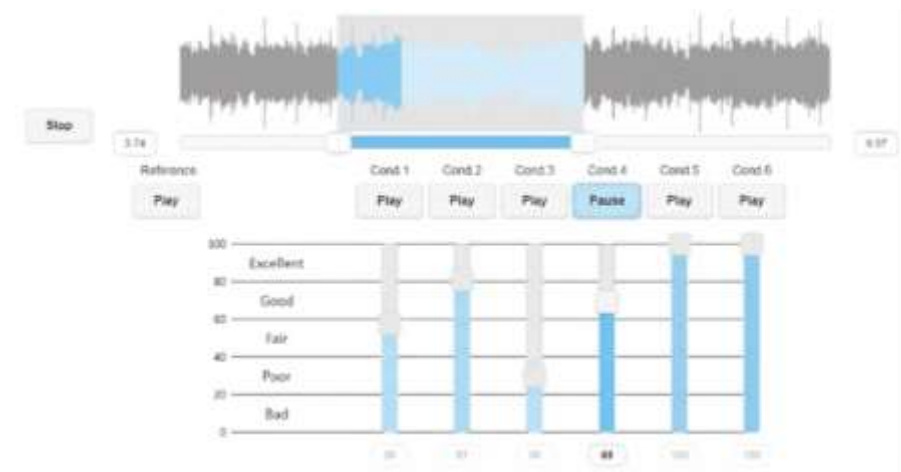
$$SBS_A = \frac{n_A}{N}, \quad SBS_B = \frac{n_B}{N}, \quad SBS_{unknown} = \frac{n_{unknown}}{N}$$

где N – количество оценок, n_A – количество голосов за вариант A , n_B – количество голосов за вариант B , $n_{unknown}$ – количество голосов за вариант «не знаю»

Субъективные оценки: *MUSHRA*

MUSHRA (*Multiple Stimuli with Hidden Reference and Anchor*) — метод проведения теста для оценки качества аудио

- Образец оценивается по шкале от 0 до 100
- В список оцениваемых примеров подмешивается оригинал (*Hidden Reference*)
- Добавляются якоря (*Anchor*) – заведомо плохие образцы для калибровки



Субъективные оценки

Достоинства	Недостатки
Хорошо интерпретируются	Невоспроизводимые результаты
Можно сделать свою шкалу	Дорого

Объективные оценки: *WER* и *CER*

WER (Word Error Rate) измеряет количество ошибок в синтезированной речи на уровне слов по сравнению с эталонным текстом:

$$WER = \frac{S + I + D}{N},$$

где N – количество слов в эталонном тексте, S – количество замен (***substitutions***), I – количество вставок (***insertions***), D – количество удалений (***deletions***)

CER (Character Error Rate) измеряет количество ошибок в синтезированной речи на уровне символов по сравнению с эталонным текстом:

$$CER = \frac{S + I + D}{N},$$

где N – количество символов в эталонном тексте, S – количество замен (***substitutions***), I – количество вставок (***insertions***), D – количество удалений (***deletions***)

Объективные оценки: *MCD*

MCD (Mel – Cepstral Distortion) измеряет различия между двумя последовательностями *MFCC* коэффициентов, полученных из синтезированной и эталонной речи:

$$MCD = \frac{10}{\ln 10} \sqrt{2 \sum_{i=1}^N \sum_{k=1}^M (c_{ik} - \hat{c}_{ik})^2}$$

где c_{ik} – *MFCC* коэффициенты эталонной записи, $\hat{c}_{ik}$ – *MFCC* коэффициенты синтезированной записи, N – количество фреймов, M – количество *MFCC* коэффициентов

Объективные оценки: *UTMOS* и *NISQA*

UTMOS (*University of Tokyo*) и *NISQA* (*Non – Intrusive Speech Quality Assessment*) – нейросетевые модели, которые автоматически предсказывают *MOS*

UTMOS специализируется на оценке естественности (*naturalness*) синтеза

NISQA оценивает общее качество аудио и наличие искажений

Объективные оценки: *SSIM*

SSIM (*Speaker Similarity*) измеряет степень сходства синтезированного и целевого голосов:

$$SSIM = \frac{E_A \cdot E_B}{||E_A|| \cdot ||E_B||}$$

где E_A и E_B — эмбединги эталонного и синтезированного голосов

Если голоса идентичны, то $SSIM \rightarrow 1$

Объективные оценки: *FAD*

FAD (Flechet Audio Distance) – нейросетевая метрика, которая оценивает общее качество аудио, сравнивая распределения признаков

- С помощью предобученной модели извлекаются эмбединги
- Для набора эталонных данных рассчитываются среднее μ_b и матрица ковариаций Σ_b
- Аналогично для синтезированных данных рассчитываются μ_e и Σ_e
- Вычисляется расстояние Фреше:

$$FAD = \|\mu_e - \mu_b\|^2 + Tr(\Sigma_b + \Sigma_e - 2\sqrt{\Sigma_b \Sigma_e})$$

Чем меньше *FAD*, тем больше распределение синтезированной речи похоже на реальное

Объективные оценки: RTF и $Latency$

RTF и $Latency$ оценивают производительность модели, что важно для *real – time* систем

RTF (*Real – Time Factor*) – отношение времени, затраченного на синтез, к длительности полученного аудио:

$$RTF = \frac{T_{inference}}{T_{audio}}$$

$RTF < 1$ – синтез быстрее реальной речи

$RTF = 1$ – скорость синтеза совпадает со скоростью речи

$Latency$ (*задержка*) – время от момента отправки запроса до получения первых байт аудио

Объективные оценки

Достоинства	Недостатки
Воспроизводимые результаты	Часто оценивают одну характеристику
Дёшево	Не коррелируют с реальной оценкой

Данные

Данные

Название	Язык	Часы	Частота дискретизации, Гц	Количество голосов
<i>LJ Speech</i>	<i>En</i>	<i>24</i>	<i>22050</i>	<i>1</i>
<i>LibriTTS</i>	<i>En</i>	<i>1000</i>	<i>24000</i>	<i>≈ 109</i>
<i>VCTK</i>	<i>En</i>	<i>44</i>	<i>48000</i>	<i>≈ 1100</i>
<i>Emilia</i>	<i>MultiLang</i>	<i>100000+</i>	<i>24000</i>	<i>—</i>
<i>HiFi TTS</i>	<i>En</i>	<i>36000</i>	<i>44100</i>	<i>—</i>
<i>Natasha</i>	<i>Ru</i>	<i>20</i>	<i>22050</i>	<i>1</i>
<i>Ruslan</i>	<i>Ru</i>	<i>31</i>	<i>44100</i>	<i>1</i>
<i>Balalaika</i>	<i>Ru</i>	<i>2000</i>	<i>-</i>	<i>—</i>

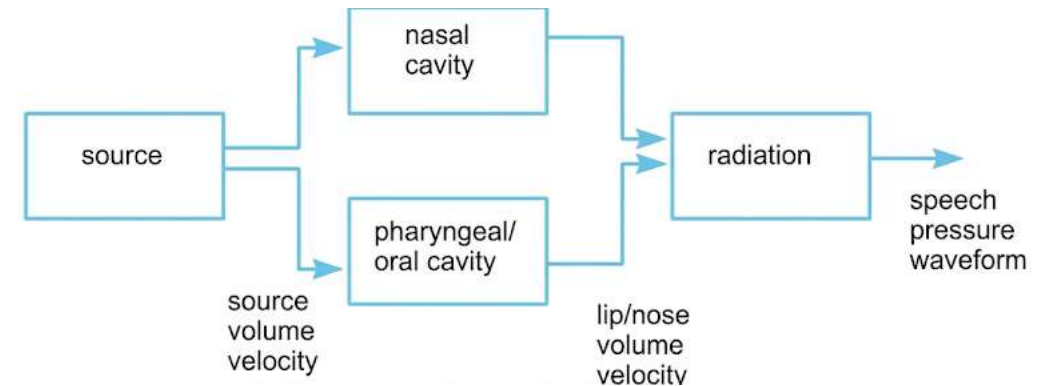
Виды синтеза речи

Формантный синтез

Суть метода: моделирование речевого тракта как набора резонаторов, каждый из которых отвечает за формирование определенной форманты

Формантный синтезатор:

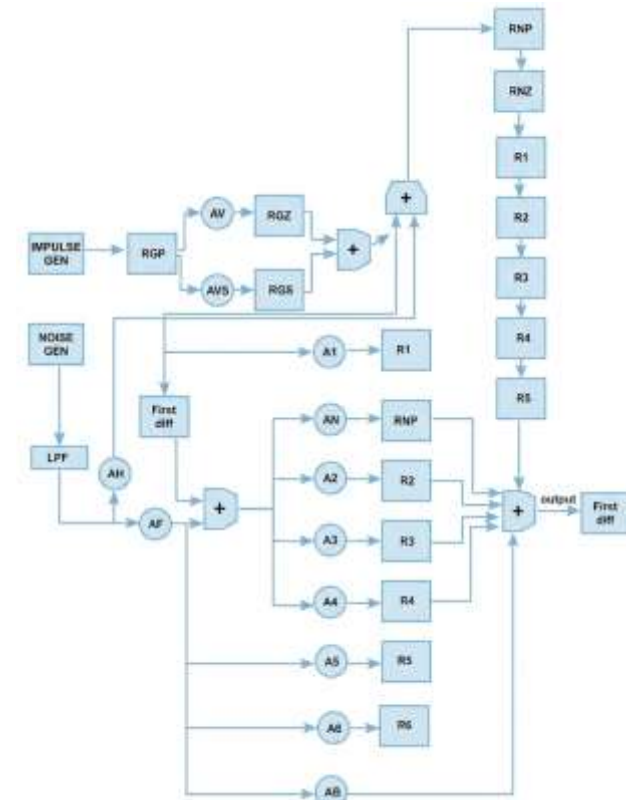
- Источник генерирует периодические сигналы для звонких звуков и шум для глухих
- Речевой тракт из нескольких резонаторов моделирует полости рта и носа
- Излучатель моделирует выход звука



Формантный синтез

Синтезатор Клатта (Klatt Synthesizer) моделирует речевой тракт и характеристики речи, используя цепочки фильтров и резонаторов

Может работать как в последовательной, так и в параллельной конфигурации



Формантный синтез

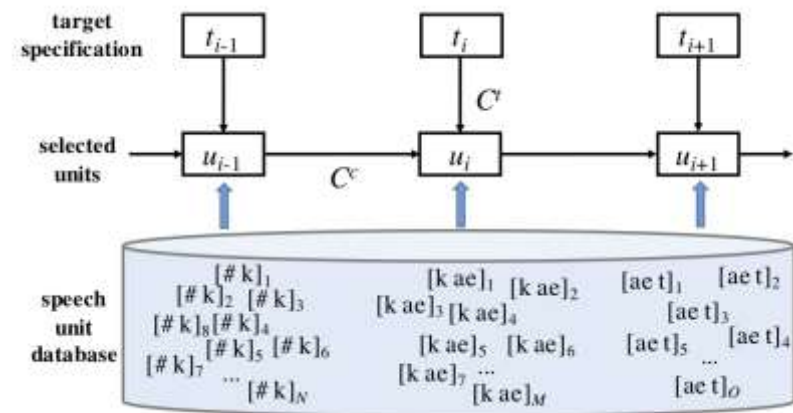
Преимущества	Недостатки
Легко понять связь между параметрами модели и характеристиками речи	Модель не учитывает все нюансы реального речевого тракта
Возможность независимого управления каждой формантой	Трудность в создании плавных переходов между звуками
Неплохая разборчивость	Неестественное звучание

Конкатенативный синтез

Идея:

Будем создавать речь путём склеивания ранее записанных фрагментов речи

- Создаём базу звуков
- Выбираем наиболее подходящие фрагменты
- Склеиваем выбранные фрагменты



Конкатенативный синтез

Преимущества	Недостатки
Хорошая разборчивость	Необходимо большое количество записанных образцов речи
Голос звучит естественно и похож на голос актёра	Ограниченная естественность и эмоциональность
Можно настраивать качество и сложность синтеза	Проблемы с плавностью звучания

Статистический параметрический синтез

Идея Statistical Parametric Speech Synthesis (SPSS):

- Будем генерировать некоторые акустические параметры речи
- С помощью специальных алгоритмов будем восстанавливать речь

Статистический параметрический синтез

Модуль анализа текста

Подготавливает текст:
нормализует, делает
сегментацию слов,
преобразует графемы в
фонемы

Акустическая модель

Связывает лингвистические
особенности текста с
акустическими параметрами
речи

Вокодер

Восстанавливает звуковой
сигнал на основе
сгенерированных параметров

Статистический параметрический синтез

Достоинства	Недостатки
Возможность изменять параметры синтеза для достижения желаемого звучания	Наличие большого количества артефактов
Требуется меньший объем записей для обучения	Речь звучит неестественно и механически

Нейронный синтез

Долгое время доминирующим подходом в *SPSS* были скрытые марковские модели (*Hidden Markov Models, HMM*)

С развитием нейросетей *HMM* стали постепенно уступать им место

Идея осталась та же – предсказывать акустические признаки из лингвистических



ОСНОВЫ ЛИНГВИСТИКИ

ОСНОВЫ ЛИНГВИСТИКИ

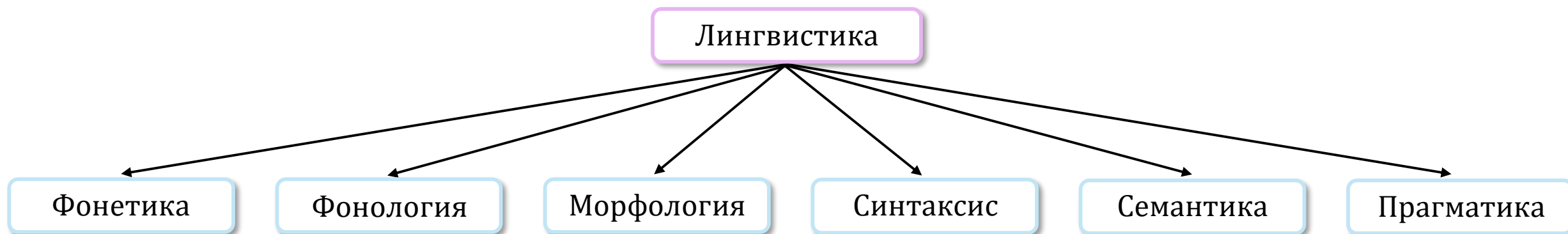
Язык – это используемая людьми система коммуникации

Язык бывает:

- Устный
- Письменный
- Жестовый

Речь – это форма общения, которая является способом выражения устного языка

Язык и речь изучает **лингвистика**, которая охватывает несколько разделов



Фонетика и фонология

Фонетика занимается изучением звуков речи с точки зрения их физических свойств:

- Как звуки производятся
- Как звуки распространяются
- Как звуки воспринимаются

Цель фонетики – дать точное описание звуков речи вне зависимости от языка

Фонология изучает, как звуки объединяются в более крупные единицы (слоги, слова) и как при этом создаются разные значения

Фонема – это минимальная единица языка, которая позволяет различать смысл слова

Морфология и синтаксис

Морфология изучает структуру слов, как слова образуются из более мелких частей – морфем

Морфема – это наименьшая часть слова, которая несёт определённый смысл

Синтаксис изучает структуру предложения, как слова соединяются друг с другом, чтобы получать осмысленные высказывания

Семантика и прагматика

Семантика изучает значения слов и их комбинации

Прагматика исследует, как язык используется в конкретных контекстах и как люди интерпретируют высказывания

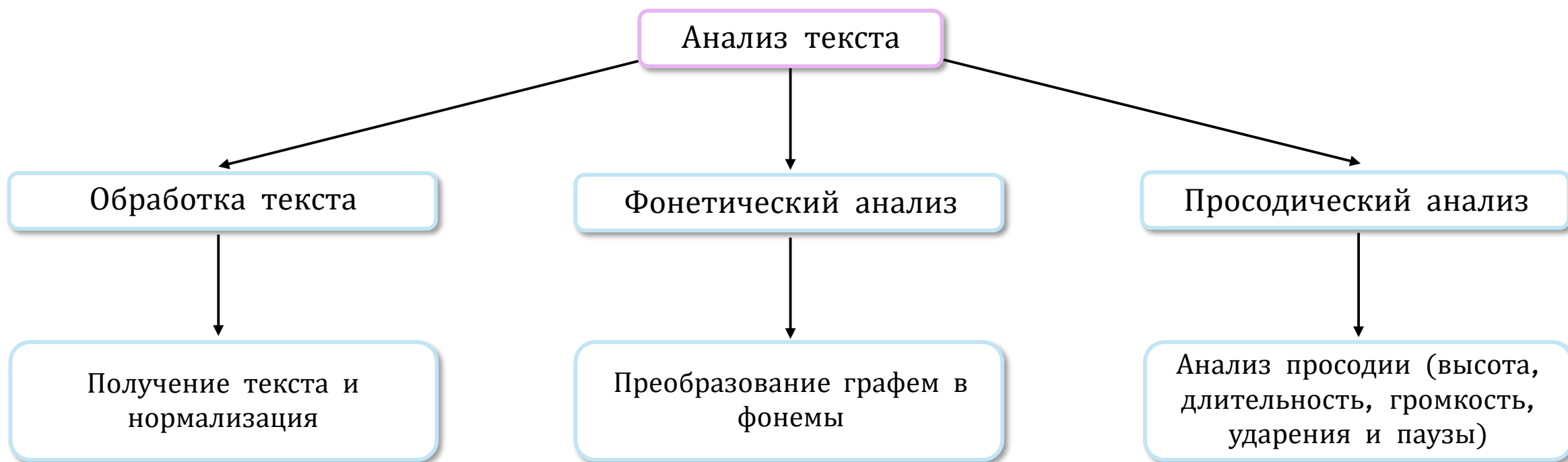
ОСНОВЫ ЛИНГВИСТИКИ

Раздел	Объект исследования	Основная единица	Важность для <i>TTS</i>
Фонетика	Все человеческие звуки	Форма сигнала	★ ★ ★ ★ ★
Фонология	Классифицированные звуки	Фонема	★ ★ ★ ★ ★
Морфология	Слова	Морфема	★ ★ ★ ★
Синтаксис	Предложения	Слово	★ ★ ★
Семантика	Значения	Морфема, слово, словосочетание, предложение	★ ★
Прагматика	Использование языка	Морфема, слово, словосочетание, предложение, контекст	★

Анализ текста

Анализ текста

Анализ текста нужен для преобразования входного текста в лингвистические признаки, которые содержат богатую информацию о произношении и просодии



Обработка текста: нормализация

Нормализация переводит текст из письменной формы в разговорную

Аббревиатуры и акронимы

КВН, МРТ и Вуз, ГОСТ

Сокращения

Респ. Татарстан, ул. Чехова

Числовые форматы и валюта

8 июля $\Rightarrow$ восьмое июля, 25 \$ $\Rightarrow$ двадцать пять долларов

Формулы

$x^2 \Rightarrow$ икс в квадрате

Транскрипция латиницы

You'll never walk alone $\Rightarrow$ Юл нэвэ волк элоун

Обработка текста: лингвистический анализ

Лингвистический анализ нужен для извлечения структурной и семантической информации из предложений

Определение типа предложения

Сегодня хорошая погода. и Сегодня хорошая погода?

Токенизация и сегментация

我爱自然语言处理 => “我“, “爱“, “自然语言处理“

Указание частей речи

Она сегодня пила. и У меня в руке пила.

Фонетический анализ

Фонетический анализ предоставляет модели информацию о том, как следует произносить слово

Разбор омографов

зАмок и замОк, хлОпок и хлопОк, бЕлки и белкИ

Перевод графем в фонемы

Knight ⇒ naɪt, *Thought* ⇒ θɔ:t

Просодический анализ

Просодический анализ необходим, чтобы речь звучала естественно

Паузы, ударения и интонации

Сегодня прекрасный день? ⇒ Сегодня прекрасный^ день? @

Длительность, высота, громкость

Сегодня ⇒ [*pitch* = 230 Гц, *Speed* = 10, *Volume* = 30]

Акустические модели

Tacotron, 2017

Проблемы прошлых моделей:

- Состояли из множества независимых модулей
- Ошибки каждого модуля накапливались
- Сложно создавать признаки
- Много эвристик и ручных настроек

Авторы **Tacotron** предложили учить модель напрямую на парах $\langle text, audio \rangle$:

- Вход – буквы или фонемы
- Выход – спектрограмма

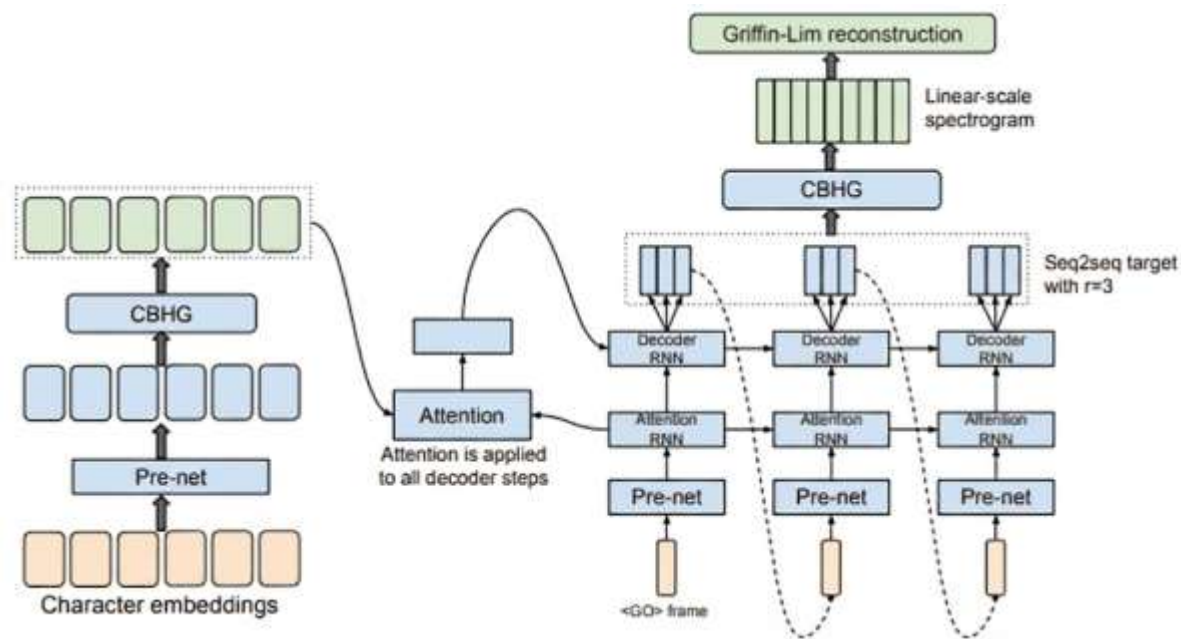
Это позволило с нуля обучать без сложных лингвистических признаков

Архитектура Tacotron

Основная идея:

- Архитектура *RNN* с механизмом внимания
- Предсказывает кадры (*frames*) спектрограммы

- *Encoder* извлекает скрытые представления текста
- *Decoder* + *Attention* предсказывает мел-спектрограмму
- Алгоритм Гриффин-Лима восстанавливает аудио

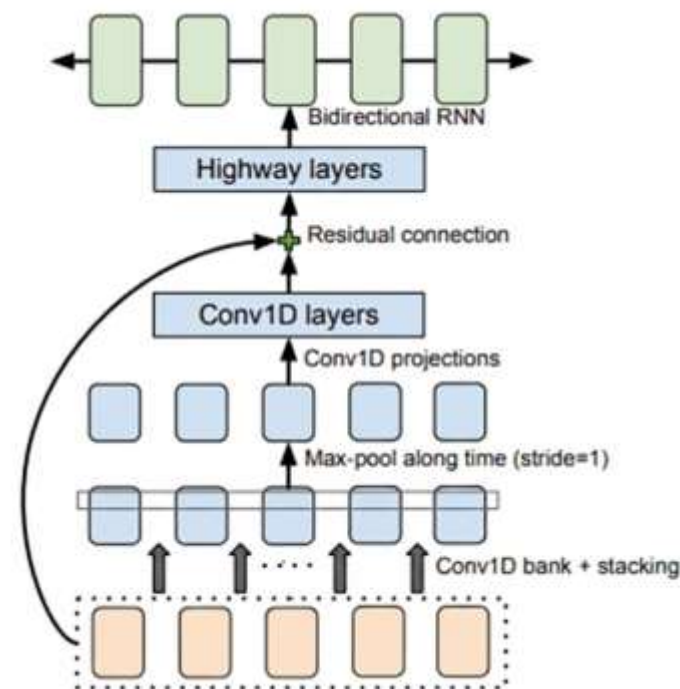


Блок CBHG

CBHG (*Convolution Bank + Highway + GRU*) – блок для извлечения признаков из последовательностей

CBHG:

- **Банк 1 – D** свёрток для захвата локального контекста
- **Max Pooling** для локальной инвариантности
- **Highway Network** для извлечения высокоуровневых признаков
- **Bidirectional RNN** для учёта контекста в обоих направлениях



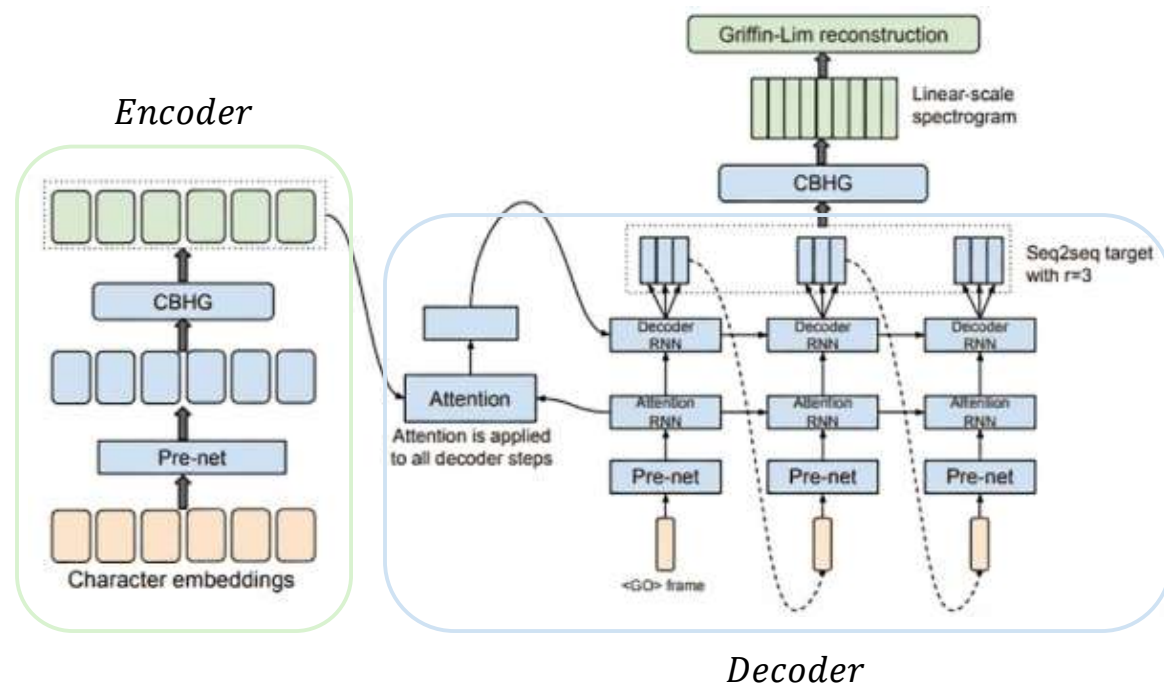
Encoder и Decoder

Encoder:

- *Pre – Net* улучшает сходимость
- Модуль **CBHG** преобразует выходы *Pre – Net* в скрытое представление

Decoder:

- *Content – based tanh attention*
- Стек **GRU** с *residual connection*
- Обучается предсказывать 80 полос мел-спектрограммы



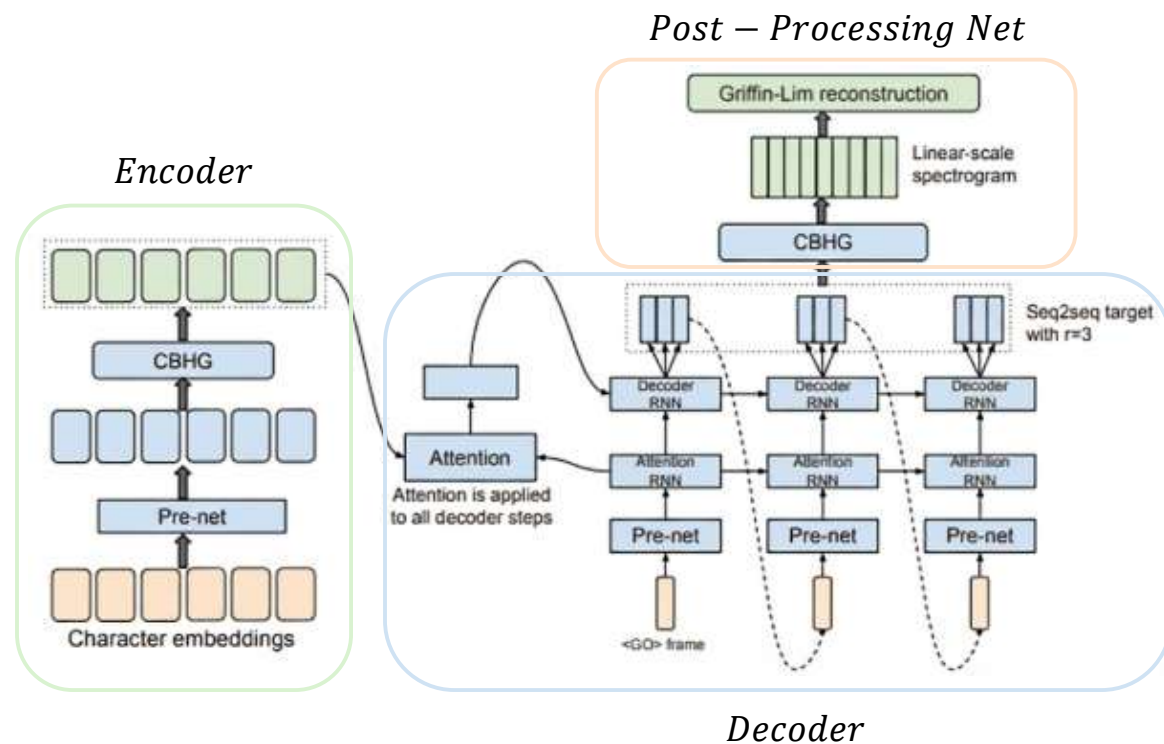
Трюки Tacotron

Декодер:

- Предсказывает несколько кадров мел-спектрограммы за один шаг ($r = 2$)
- Это уменьшает размер модели и время обучения/инференса, стабилизирует обучение

Post – Processing Net:

- Преобразует мел-спектрограмму в спектрограмму для вокодера
- Является *bidirectional*



Результаты Tacotron

- Модель училась на внутреннем наборе данных ≈ 24.6 часов речи
- В качестве вокодера использовался алгоритм Гриффин-Лима

	mean opinion score
Tacotron	3.82 ± 0.085
Parametric	3.69 ± 0.109
Concatenative	4.09 ± 0.119

*Generative adversarial network
or variational auto-encoder*



*The quick brown fox jumps over
the lazy dog*



Tacotron 2, 2017

Идея:

- Объединить 2 лучшие архитектуры *Tacotron* и *WaveNet*

Проблемы предшественников:

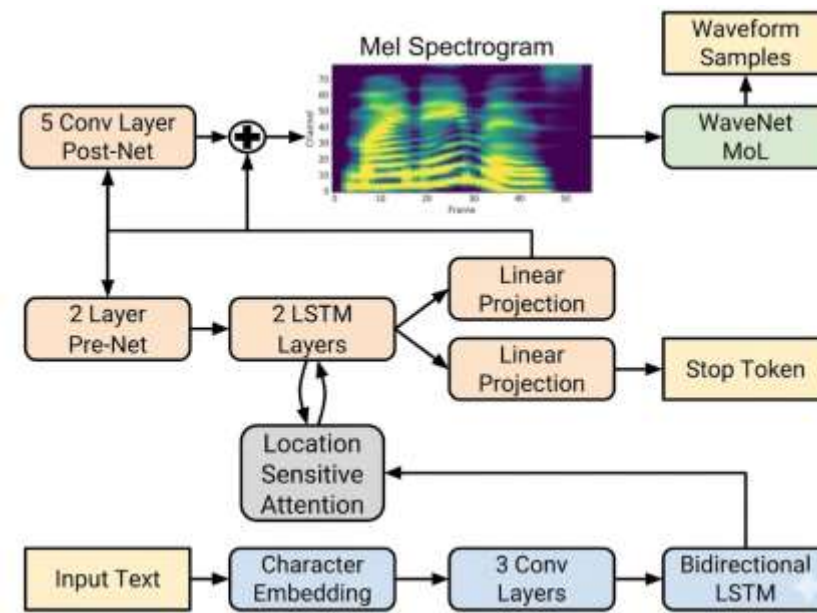
- *Tacotron* использовал алгоритм Гриффин-Лима, который давал артефакты и роботизированный звук
- *WaveNet* требовал сложные лингвистические признаки

Решение:

Будем предсказывать мел-спектрограмму из текста и озвучивать её с помощью вокодера

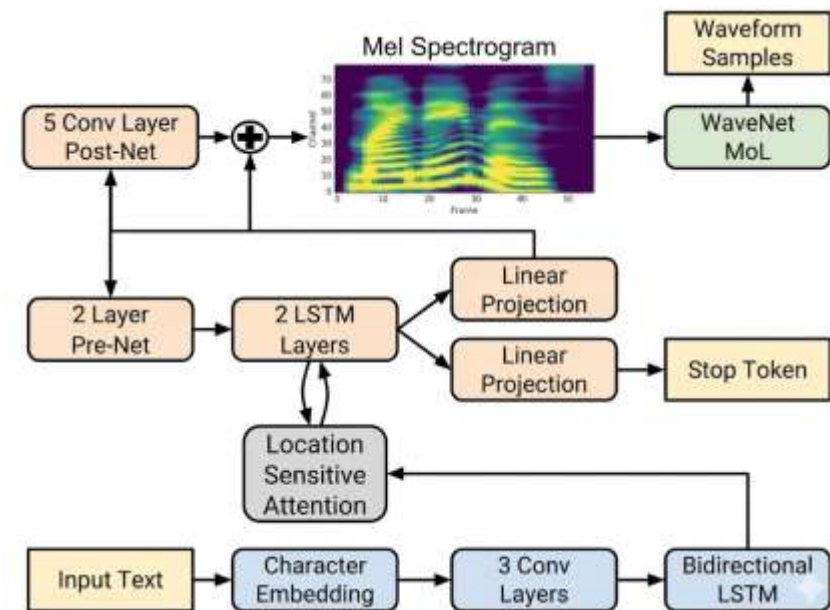
Spectrogram Prediction Network

- Авторы отказались от *CBHG* и используют *свёрточные слои* и *LSTM*
- *Encoder* состоит из 3 свёрточных слоёв и *Bidirectional LSTM*
- *Location Sensitive Attention* учитывает предыдущие шаги внимания, что предотвращает пропуски слов или их бесконечное повторение



Spectrogram Prediction Network

- **2 LSTM Layers** одновременно обрабатывает акустическую и текстовую информацию
- На каждом шаге **LSTM** генерирует «черновик» кадра спектрограммы и вероятность окончания фразы
- *Post – Net* предсказывает недостающие детали кадра спектрограммы
- Финальный результат – сумма **LSTM** и *Post – Net*



Результаты Tacotron 2

- Модель училась на внутреннем наборе данных ≈ 24.6 часов речи
- Нейросети обучались отдельно
- *WaveNet* обучался на предсказанных, а не эталонных мел-спектрограммах

System	MOS
Parametric	3.492 ± 0.096
Tacotron (Griffin-Lim)	4.001 ± 0.087
Concatenative	4.166 ± 0.091
WaveNet (Linguistic)	4.341 ± 0.051
Ground truth	4.582 ± 0.053
Tacotron 2 (this paper)	4.526 ± 0.066

*Generative adversarial network
or variational auto-encoder*



*The quick brown fox jumps over
the lazy dog*



Transformer TTS, 2018

Проблемы Tacotron 2:

- *RNN* не могут распараллеливать вычисления
- Сложно удерживать длинный контекст

Решение:

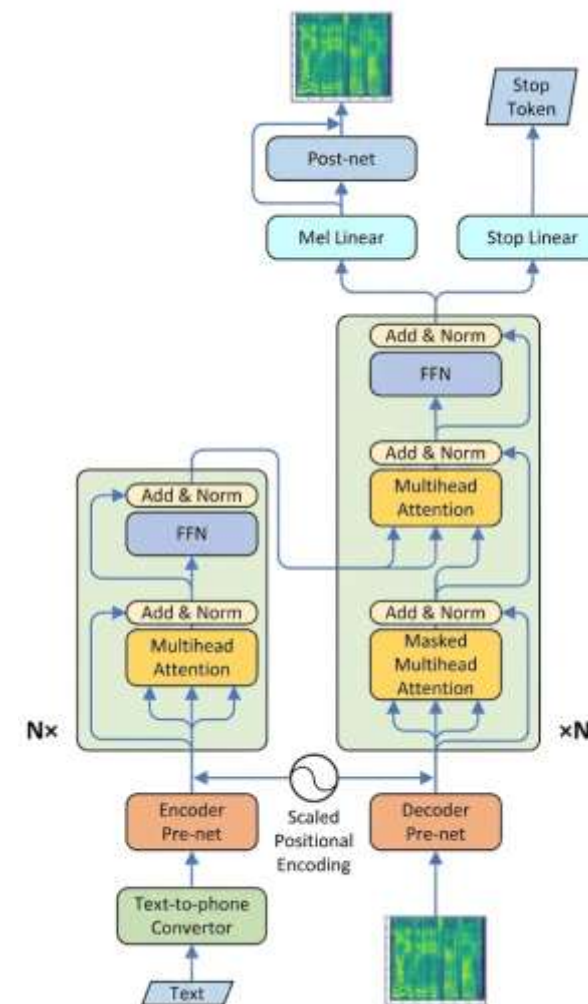
Заменим *RNN* на *Transformer* с *Multi – Head Attention*

Архитектура *Transformer TTS*

- Модель работает с фонемами
- **Encoder Pre – Net** превращает фонемы в эмбединги
- Поскольку масштаб данных у текста и спектрограммы разный, авторы используют ***position encoding*** с обучаемым параметром α :

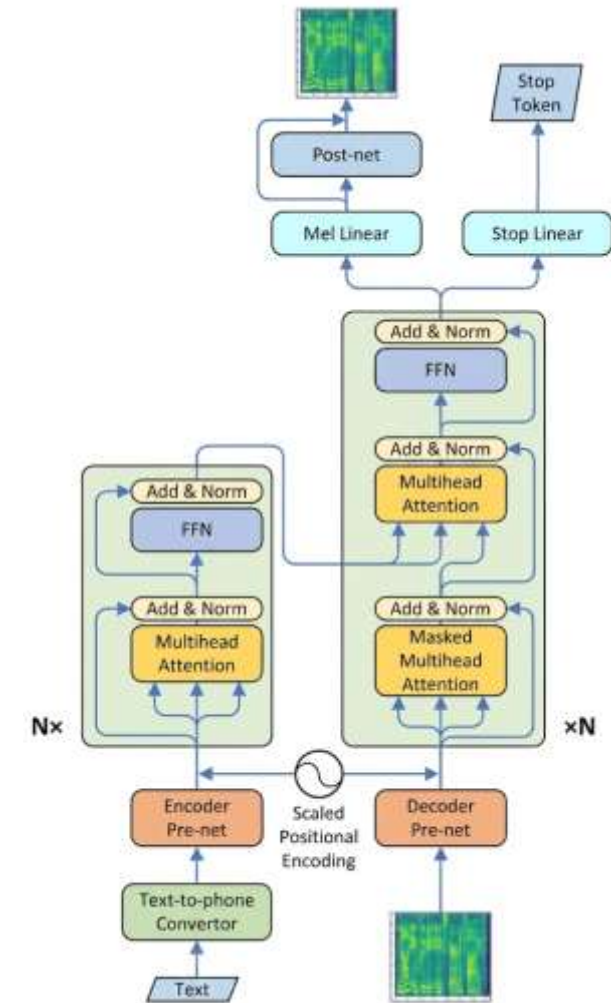
$$x = \text{PreNet}(\text{input}) + \alpha \cdot PE$$

- **Encoder** представляет собой N блоков энкодера трансформера



Decoder Transformer TTS

- Фонемы живут в адаптивном пространстве, а спектрограммы – в фиксированном
- Задача **Decoder Pre – Net** спроецировать звук в то же подпространство, где живут фонемы, чтобы **attention** мог вычислить сходство
- Как и в **Tacotron 2** данные после декодера разделяются на 2 потока
- **Mel Linear** делает грубое предсказание спектрограммы
- **Post – Net** вычисляет остаток и добавляет к прогнозу
- **Stop Linear** предсказывает вероятность окончания речи



Результаты *Transformer TTS*

- Модель училась на внутреннем наборе данных ≈ 25 часов речи
- В качестве вокодера используется *WaveNet*
- Один шаг обучения в 4 раза быстрее *Tacotron 2*

System	MOS	CMOS
Tacotron2	4.39 ± 0.05	0
Our Model	4.39 ± 0.05	0.048
Ground Truth	4.44 ± 0.05	-

*Flooding is likely in some
parishes of southern Louisiana*



*In Texas, hospitals have passed along
higher costs to local taxpayers*



FastSpeech, 2019

Проблемы прошлых моделей:

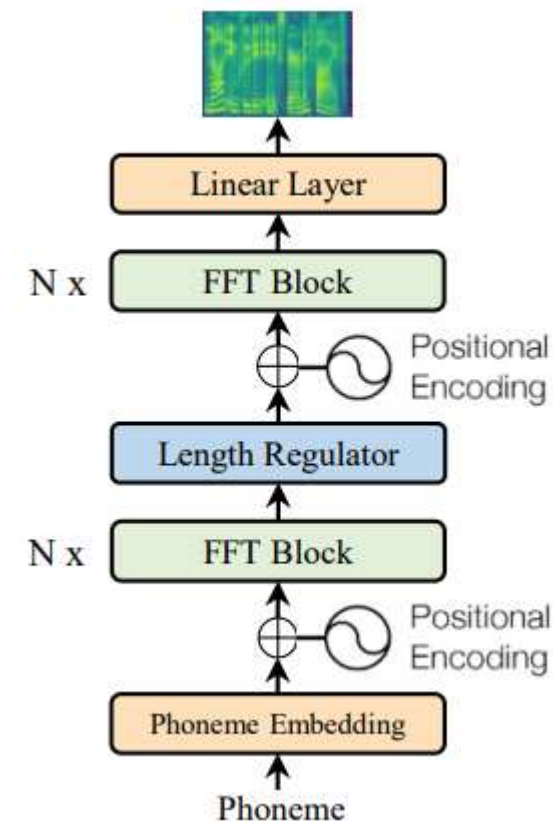
- Низкая скорость генерации из-за авторегрессии
- Из-за *Soft Attention* возникают пропуски слов, повторы, бормотания
- Невозможно управлять скоростью речи

Идея:

- Сделаем неавторегрессионную модель
- Будем параллельно генерировать все кадры мел-спектрограммы

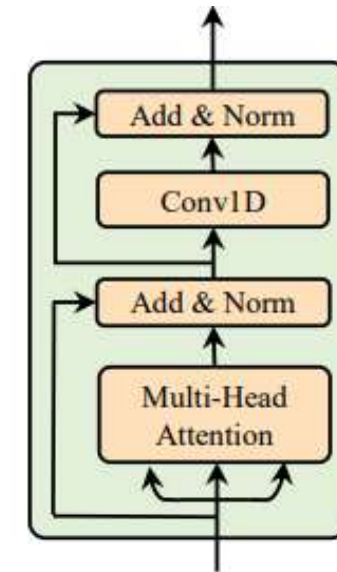
Архитектура *FastSpeech*

- Модель состоит из стека ***Feed Forward Transformer (FFT)*** блоков
- На вход приходят фонемы
- ***Length Regulator*** вычисляет длительность каждой фонемы
- За один проход получаем готовую спектрограмму



Feed- Forward Transformer

- **FFT** – измененный блок трансформера, сделанный для генерации речи
- **Multi – Head Attention** отвечает за глобальный контекст
- Вместо полносвязной сети используется свёртка для улавливания локальных зависимостей



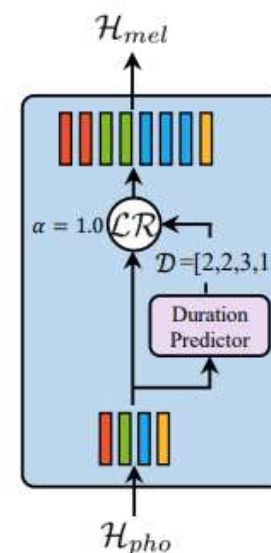
Length Regulator

- Длина аудио намного больше длины текста
- В авторегрессионных моделях их связывает **Cross – Attention**, который часто ошибается

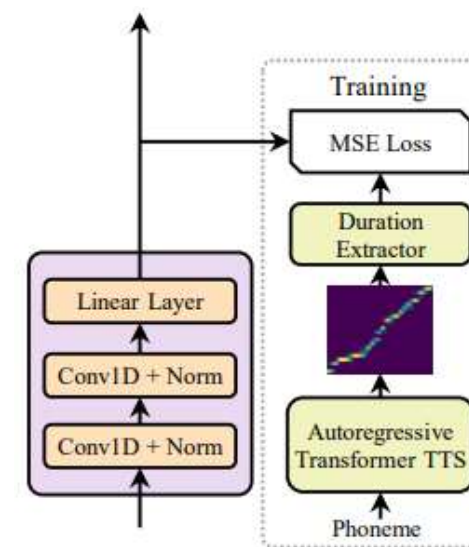
- Авторы предложили заменить **Cross – Attention** на **Length Regulator**
- **Length Regulator** для каждой фонемы предсказывает длительность
- Каждая фонема дублируется d раз:

$$H_{mel} = LR(H_{pho}, D, \alpha),$$

где d – длительность фонемы, H_{pho} – скрытое состояние фонем, D – длительность, α – коэффициент скорости



Length Regulator



Duration Predictor

Результаты *FastSpeech*

- Модель училась на датасете *LJSpeech*
- Генерация мел-спектрограммы в 270 раз быстрее *Transformer TTS*
- С помощью параметра α можно ускорять или замедлять речь

Method	MOS
<i>GT</i>	4.41 ± 0.08
<i>GT (Mel + WaveGlow)</i>	4.00 ± 0.09
<i>Tacotron 2 [22] (Mel + WaveGlow)</i>	3.86 ± 0.09
<i>Merlin [28] (WORLD)</i>	2.40 ± 0.13
<i>Transformer TTS [14] (Mel + WaveGlow)</i>	3.88 ± 0.09
<i>FastSpeech (Mel + WaveGlow)</i>	3.84 ± 0.08

was executed on a gibbet in front of his victim's house



Speed: 0.75x



Speed: 1x



Speed: 1.25x

FastSpeech 2, 2020

Проблемы FastSpeech:

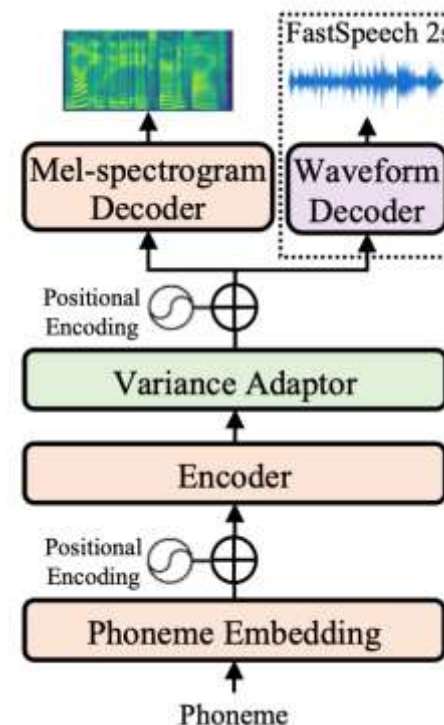
- Сначала учится авторегрессионная модель, а затем происходит дистилляция знаний
- Мел-спектрограммы, полученные от учителя, упрощены и содержат меньше информации
- Длительность фонем не всегда точная
- Одному тексту могут соответствовать разные варианты произношения (скорость, интонация, громкость) и модели сложно их предсказывать

Идея:

- Откажемся от модели-учителя
- Введем *Variance Adaptor* для получения дополнительной информации о произношении

Архитектура *FastSpeech 2*

- **Encoder** преобразует эмбединги фонем в последовательности скрытых состояний
- **Variance Adaptor** обогащает последовательность данными о длительности, высоте и энергии
- **Decoder** преобразует подготовленные данные в мел-спектрограмму
- В основе лежат **Feed – Forward Transformer (FFT)** блоки
- Есть версия **FastSpeech 2s** сразу для генерации аудио



Variance Adaptor

Variance Adaptor добавляет информацию о просодии к скрытой последовательности фонем

Duration Predictor:

- Использует **Montreal Forced Alignment (MFA)** вместо *Attention Maps*, что дает более точные границы фонем

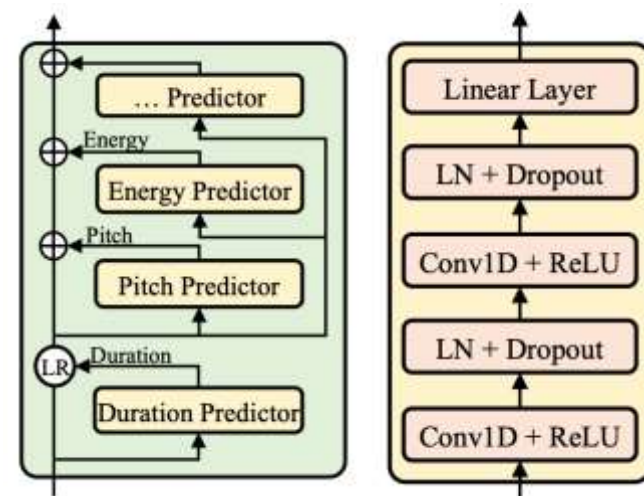
Pitch Predictor:

- Использует непрерывное вейвлет преобразование (CWT) для разложения контура высоты в спектрограмму, что позволяет лучше моделировать просодию по сравнению с прямым предсказанием

- Значения квантуются и добавляются к скрытым состояниям

Energy Predictor:

- Вычисляется как L_2 -норма амплитуды кадров *STFT*, что отражает громкость речи



Результаты *FastSpeech 2*

- Модель училась на датасете *LJSpeech*
- Генерация мел-спектрограммы в 48 раз быстрее *Transformer TTS*
- Удаление информации о высоте или энергии значительно снижает качество

Method	MOS
<i>GT</i>	4.30 ± 0.07
<i>GT (Mel + PWG)</i>	3.92 ± 0.08
<i>Tacotron 2 (Shen et al., 2018) (Mel + PWG)</i>	3.70 ± 0.08
<i>Transformer TTS (Li et al., 2019) (Mel + PWG)</i>	3.72 ± 0.07
<i>FastSpeech (Ren et al., 2019) (Mel + PWG)</i>	3.68 ± 0.09
<i>FastSpeech 2 (Mel + PWG)</i>	3.83 ± 0.08
<i>FastSpeech 2s</i>	3.71 ± 0.09

(a) The MOS with 95% confidence intervals.

Method	CMOS
<i>FastSpeech 2</i>	0.000
<i>FastSpeech</i>	-0.885
<i>Transformer TTS</i>	-0.235

(b) CMOS comparison.

Glow – TTS, 2020

Проблемы авторегрессионных моделей:

- Медленная генерация
- Пропускают слова или повторяют фразы, особенно на длинных предложениях

Проблемы существующих параллельных моделей:

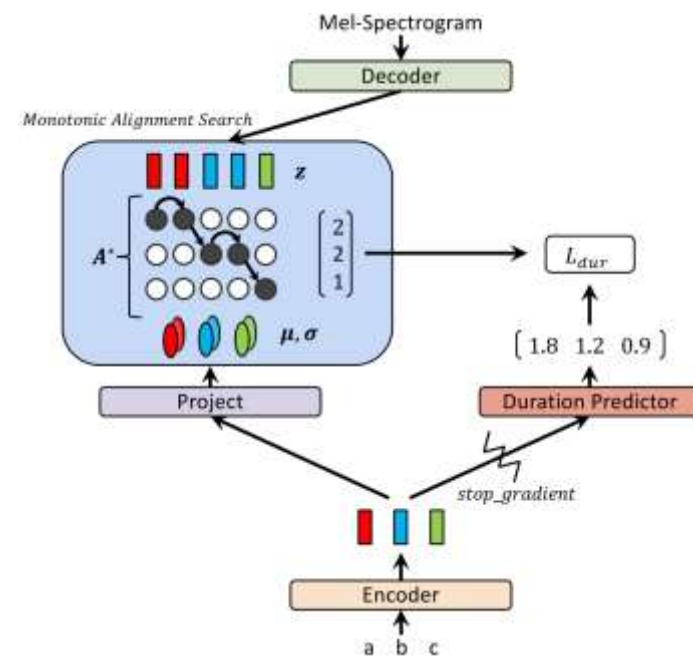
- Зависимость от учителя
- Сложный двухэтапный процесс обучения

Идея:

- Будем учить *Flow – based* модель, которая во время обучения будет находить связь между текстом и речью

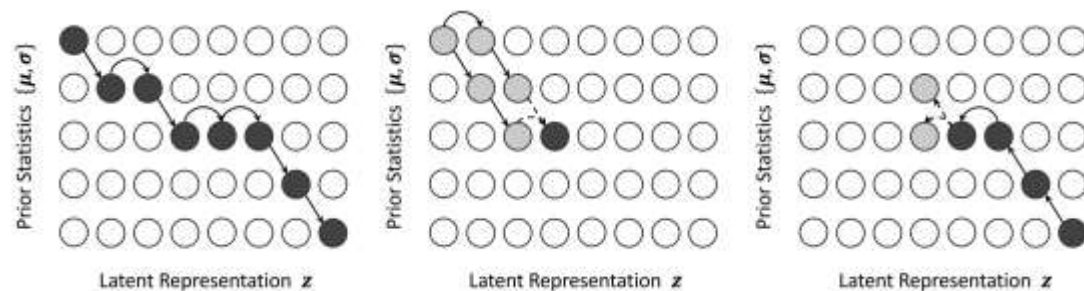
Архитектура Glow – TTS

- Модель не требует внешнего выравнивания, а делает это сама с помощью **Monotonic Alignment Search (MAS)**
- **Text Encoder** выдает параметры распределения, из которого будет сэмплироваться речь
- Вместо *absolute positional encoding* используется **relative positional encoding**
- Для каждого входного символа **Text Encoder** выдает параметры μ и σ , которые определяют априорное распределение $P_z(z|c) = \mathcal{N}(\mu, \sigma)$



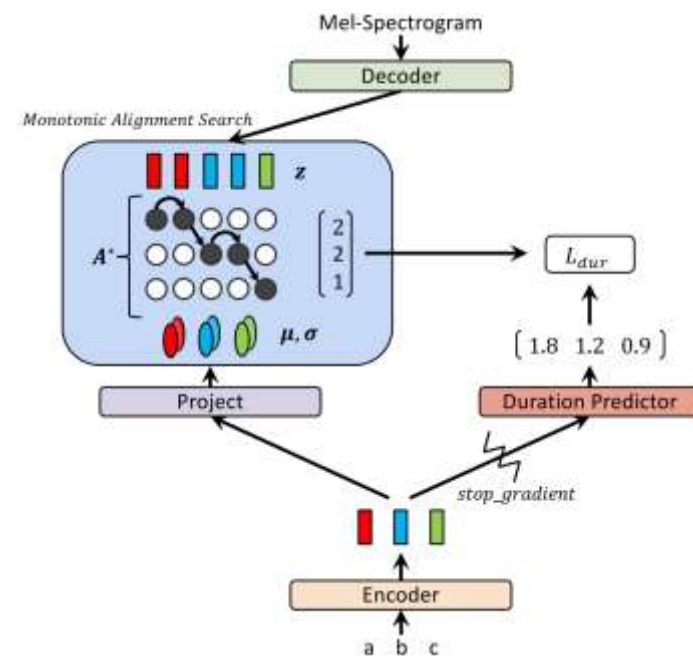
Monotonic Alignment Search

- Алгоритм ищет путь, соединяющий начало и конец фразы и может двигаться только вправо или по диагонали
- Для каждой текущей точки вычисляем максимально накопленное правдоподобие
- Делаем обратный проход и находим самый оптимальный путь
- Получаем жесткое выравнивание (*hard alignment*), где каждому фрейму аудио соответствует конкретная фонема



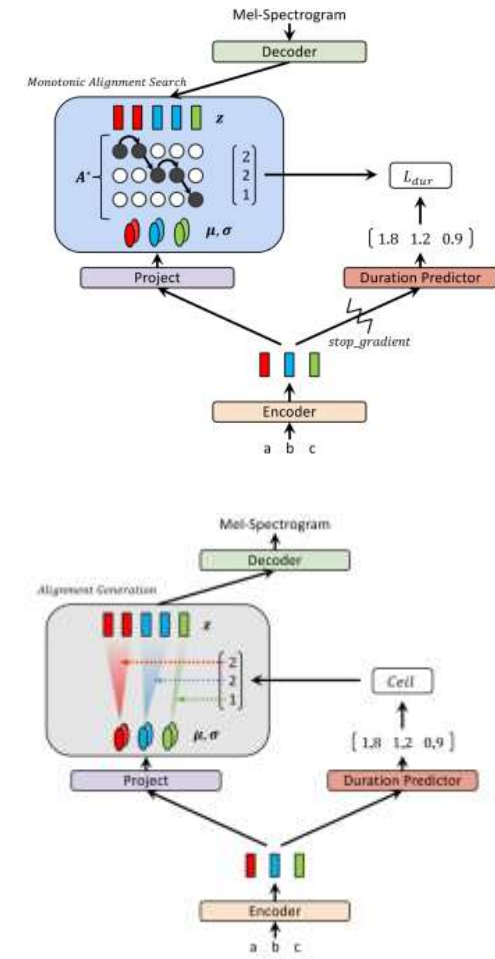
Duration Predictor

- Архитектура заимствована у *FastSpeech* – небольшая сверточная сеть
- Во время обучения благодаря *MAS* модель знает выравнивания
- *Duration Predictor* учится предсказывать длительности
- Используется *stop gradient*, чтобы ошибка от *Duration Predictor* не шла в *Text Encoder*



Flow – based Decoder

- Основан на архитектуре *WaveGlow*
- Состоит из стека одинаковых блоков *Squeeze*, *ActNorm*, *Inv 1x1 Conv*, *Affine Coupling Layers*
- Во время обучения превращаем мел-спектрограмму в базовое распределение $\mathcal{N}(\mu, \sigma)$
- На инференсе сэмплируем вектор $\mathbf{z}$ и прогоняем через поток в прямом направлении



Результаты *Glow – TTS*

- Модель училась на датасете *LJSpeech*
- Скорость в 16 раз быстрее *Tacotron 2*
- Можно управлять скоростью речи и даже интонацией
- Имеет *Multi – speaker* реализацию

Method	9-scale MOS
GT	4.54 ± 0.06
GT (Mel + WaveGlow)	4.19 ± 0.07
Tacotron2 (Mel + WaveGlow)	3.88 ± 0.08
Glow-TTS ($T = 0.333$, Mel + WaveGlow)	4.01 ± 0.08
Glow-TTS ($T = 0.500$, Mel + WaveGlow)	3.96 ± 0.08
Glow-TTS ($T = 0.667$, Mel + WaveGlow)	3.97 ± 0.08

On the seventh July, eighteen thirty-seven



$T = 0.333$



$T = 1.0$

Grad – TTS, 2021

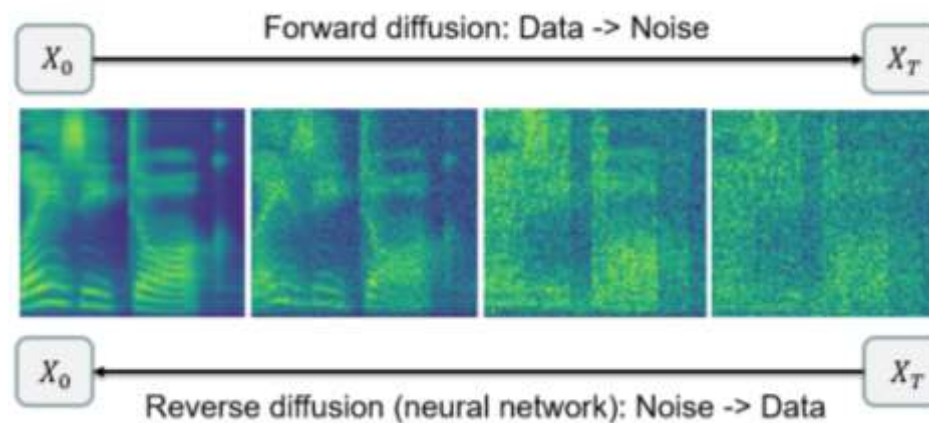
Glow – TTS:

Не требует внешнего выравнивания, а использует *MAS*

Голос звучит роботизированно и много ошибок в ударениях

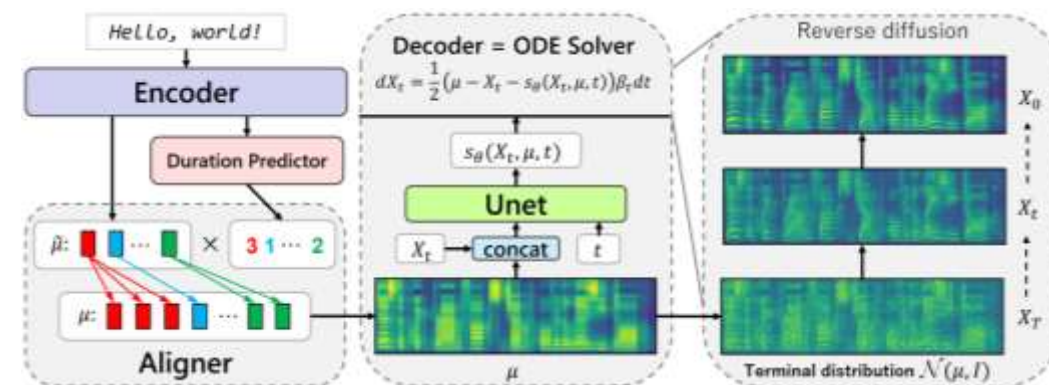
Идея:

- Обучим диффузионную модель для генерации мел-спектрограмм



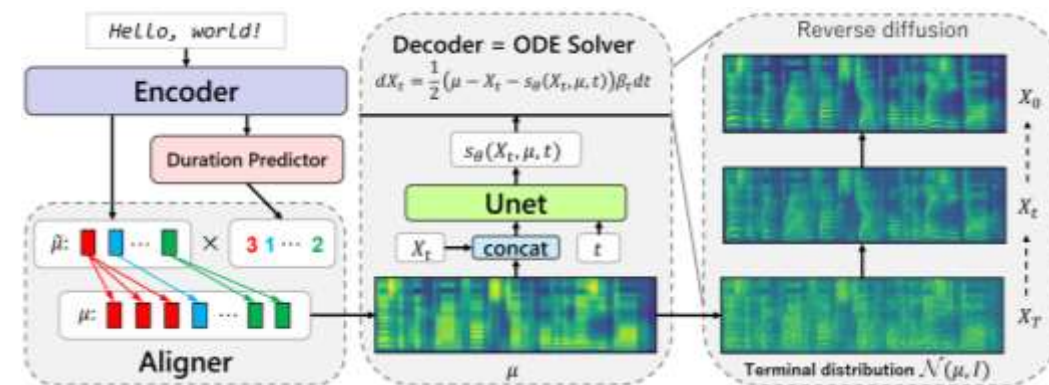
Архитектура *Grad* – TTS

- **Encoder** имеет архитектуру *Transformer*, которая превращает текст в последовательность векторов μ
- Для выравнивания используется *MAS*
- **Duration Predictor** предсказывает длительности фонем



Decoder

- **Decoder** имеет *U – Net* архитектуру
- **Encoder** дает хорошее приближение спектрограммы μ
- Добавляем шум к μ
- Учим декодер убирать этот шум и восстанавливать спектрограмму
- Начинаем генерацию с $\mathcal{N}(\mu, I)$



Результаты *Grad – TTS*

- Модель училась на датасете *LJSpeech*
- Результаты *MOS* лучше *Tacotron2*, особенно на 1000 шагах
- По сравнению с *Glow – TTS* делает меньше ошибок в ударениях и звучит менее механически

Model	Enc params ¹	Dec params	RTF	Log-likelihood	MOS
Grad-TTS-1000	7.2m	7.6m	3.663	0.174 ± 0.001	4.44 ± 0.05
Grad-TTS-100			0.363		4.38 ± 0.06
Grad-TTS-10			0.033		4.38 ± 0.06
Grad-TTS-4			0.012		3.96 ± 0.07
Glow-TTS	7.2m	21.4m	0.008	0.082	4.11 ± 0.07
FastSpeech	24.5m		0.004	—	3.68 ± 0.09
Tacotron2	28.2m		0.075	—	4.32 ± 0.07
Ground Truth	—		—	—	4.53 ± 0.06

Matcha – TTS, 2023

Проблемы диффузионных моделей:

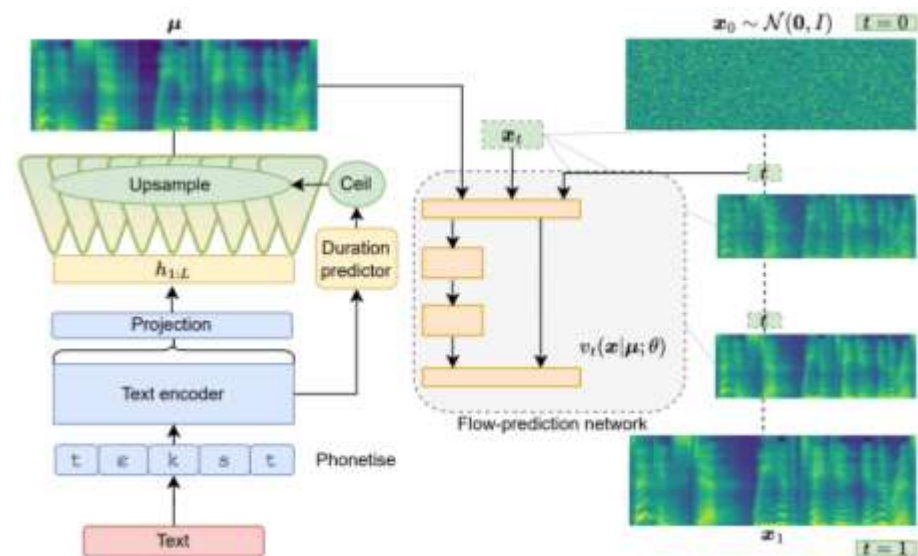
- Требуют много шагов генерации для получения высокого качества
- Обучать через *Score Matching* вычислительно затратно
- Использовать 2D свёртки довольно затратно

Идея:

- Будем использовать ***Conditional Flow Matching (CFM)*** вместо диффузии

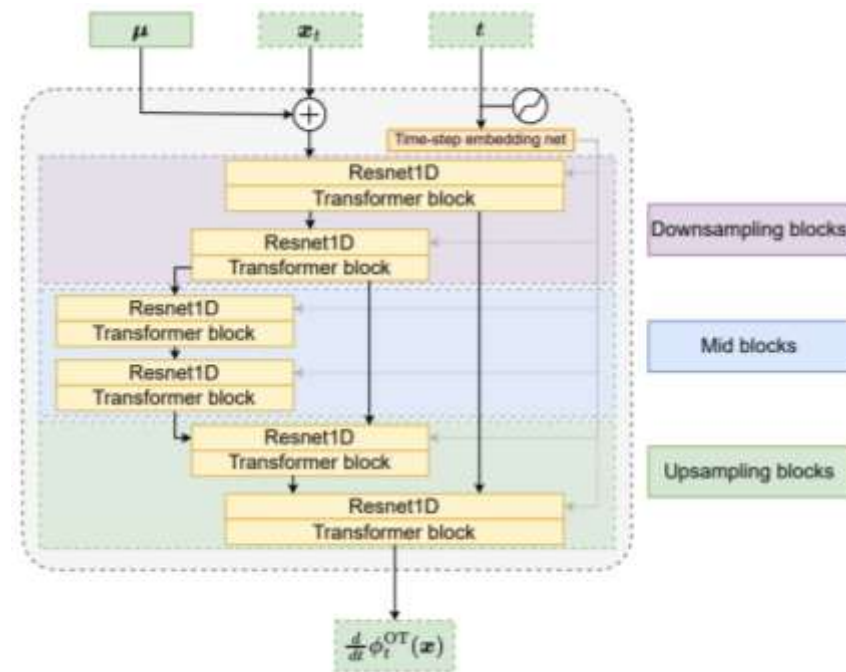
Архитектура Matcha – TTS

- *Encoder* как у *Glow – TTS* и *Grad – TTS*
- Вместо **Relative PE** используется **Rotary PE**
- Модель сама выучивать длительности с помощью **MAS**
- **Duration Predictor** предсказывает длительности фонем



Decoder

- Используются *U – Net* архитектура с 1D свёртками вместо 2D
- *Mel – Bins* выступают в роли признаков, а не как пространственное измерение
- Чередование свёрток и трансформеров
- В трансформерах используются активации *snake beta* для лучшего моделирования периодических сигналов



Результаты Matcha – TTS

- Модель училась на датасете *LJSpeech*
- Небольшая по числу параметров
- Низкий *RTF*
- На 10 шагах выдает лучшее качество

Condition	Params.	RAM	RTF ($\mu\pm\sigma$)	WER	MOS
VOC	13.9M	-	0.001 $\pm$ 0.001	1.97	4.13 $\pm$ 0.09
FS2	41.2M	6.0	0.010 $\pm$ 0.004	4.18	3.29 $\pm$ 0.09
VITS	36.3M	12.4	0.074 $\pm$ 0.083	2.52	3.71 $\pm$ 0.08
GRAD-10	14.8M	7.8	0.049 $\pm$ 0.013	3.44	3.49 $\pm$ 0.08
GRAD-4	"	"	0.019 $\pm$ 0.006	3.69	3.20 $\pm$ 0.09
GCFM-4	"	"	0.019 $\pm$ 0.004	2.70	3.57 $\pm$ 0.08
MAT-10	18.2M	4.8	0.038 $\pm$ 0.019	2.09	3.84 $\pm$ 0.08
MAT-4	"	"	0.019 $\pm$ 0.008	2.15	3.77 $\pm$ 0.07
MAT-2	"	"	0.015 $\pm$ 0.006	2.34	3.65 $\pm$ 0.08

The Secret Service believed that it was very doubtful that any President would ride regularly in a vehicle with a fixed top, even though transparent



Спасибо за внимание!